NAME :KANISHKA SHARMA

ROLL NO-26

SEC:B

Unit II Assignment: Front-End Development — HTML, CSS & JavaScript

Q1. Write CSS Flexbox code for a navbar with a logo on the left and 3 menu links on the right, all vertically centered.

Answer:

```
<nav class="navbar">
  <div class="logo">MyLogo</div>
  <div class="menu">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
</nav>
```

```
.navbar { display: flex;
justifycontent: space-between;
alignitems: center; padding:
15px;
}
```

```
.menu {
display: flex;
gap: 20px;
}
```

Q2. This button is supposed to toggle a "dark-mode" class on the body when clicked, but nothing happens. Find and fix the bug.

Answer:

Name: Kashish Chauhan
The spelling of `addEventListener` is incorrect.
Roll No.: 28

Correct code:

```
const button = document.querySelector('.toggle-btn');

button.addEventListener('click', () => {
document.body.classList.toggle('dark-mode');
});
```

Q3. Write vanilla JavaScript to validate a signup form: check that the email contains "@" and the password is at least 8 characters, before allowing submission — without using any library.

Answer:

```
<form id="signupForm">
  <input type="email" id="email" placeholder="Email">
  <input type="password" id="password" placeholder="Password">
  <button type="submit">Sign Up</button>
</form>

document.getElementById("signupForm").addEventListener("submit", function(e) {
e.preventDefault();

  const email = document.getElementById("email").value;   const  password
= document.getElementById("password").value;

  if (!email.includes("@")) {      alert("Enter
a valid email");     return;
  }

  if (password.length < 8) {     alert("Password  must
be at least 8 characters");     return;
  }

  alert("Signup successful!");
});
```

Q4. This code attaches a new scroll event listener every time `renderPage()` runs, without ever removing the old one. In 2 lines, explain why this causes a memory leak, then rewrite it correctly.

Answer:

Every call to `renderPage()` creates a new event listener.
The old listeners remain attached, causing unnecessary memory usage and repeated execution. **Correct code:**

```
function handleScroll() {
console.log(window.scrollY);
}

function renderPage() {    window.removeEventListener('scroll', handleScroll);
window.addEventListener('scroll',
handleScroll);
}

renderPage(); renderPage();
```

Q5. Write a debounced search-input handler in JavaScript that waits 400ms after the user stops typing before calling an API function `searchProducts(query)`.

Answer:

```
let timer;

const searchInput = document.querySelector("#search");

searchInput.addEventListener("input", function() {    clearTimeout(timer);

  const query = this.value;

  timer = setTimeout(() => {      searchProducts(query);
  }, 400);
});
```

Q6. This layout breaks below 480px screen width. Add a media query that fixes it for mobile, and explain in 1 line why the original CSS breaks.

Answer:

The original CSS uses fixed widths (`960px` and `300px`), which are too large for small screens.

```
.container {   display: flex;   width: 960px;
}

.card {   width:
300px;
}

@media (max-width: 480px) {
.container {     width: 100%;     flex-
direction:
column;
  }

  .card {
width: 100%;
  }
}
```

Q7. Write JavaScript code using localStorage to save and restore a user's selected theme ("light" or "dark") so it persists across page reloads.

Answer:

```
const savedTheme = localStorage.getItem("theme");

if (savedTheme) {
document.body.classList.add(savedTheme);
}

function setTheme(theme) {
document.body.classList.remove("light", "dark");
document.body.classList.add(theme);

  localStorage.setItem("theme", theme);
}

setTheme("dark");
```

Q8. This form refreshes the page on submit and loses all JS state. Rewrite the code to prevent the default behavior and submit the data using `fetch()` instead.

Answer:

```
document.getElementById("contactForm").addEventListener("submit", async function(e) {
e.preventDefault();

  const email = this.email.value;

  const response = await fetch("/api/contact", {
method: "POST",     headers: {         "Content-
Type": "application/json"
    },
    body: JSON.stringify({ email: email })
  });

  const data = await response.json();

  console.log(data);
});
```

Q9. This async function is meant to return the parsed user data, but callers receive a pending Promise instead. Identify the missing keyword and correct the code.

Answer:

The missing keyword is **`await`**.

```
async function getUser(id) {   const res =
await fetch(`/api/users/${id}`);   const data
= await res.json();

  return data;
}
```

Q10. Write CSS Grid code for a responsive image gallery with 4 columns on desktop and 2 columns on mobile, using `minmax()` and `auto-fit` instead of a media query.

Answer:

```
<div class="gallery">
  <img src="image1.jpg">
  <img src="image2.jpg">
  <img src="image3.jpg">
  <img src="image4.jpg">
  <img src="image5.jpg">
  <img src="image6.jpg">
</div>
```

```
.gallery {   display: grid;   grid-template-columns: repeat(autofit, minmax(250px,
1fr));   gap: 15px;
}
```

```
.gallery img {   width:
100%;   height:
auto;
}
```